第16章　神兵利器——optimizer trace的神奇功效

16.1　optimizer trace 简介

对于 MySQL 5.6 以及之前的版本来说，查询优化器就像是一个黑盒子，我们只能通过 EXPLAIN 语句查看到优化器最终决定使用的执行计划，却无法知道它为什么做出这样的决策。这对于一部分喜欢刨根问底的同学来说简直是灾难："我就觉得使用其他的执行方案比 EXPLAIN 语句输出的方案强，凭什么优化器做的决定与我想的不一样呢？"

在 MySQL 5.6 以及之后的版本中，设计 MySQL 的大叔很贴心地为这部分同学提供了一个 optimizer trace 的功能。这个功能可以让用户方便地查看优化器生成执行计划的整个过程。这个功能的开启与关闭由系统变量 optimizer_trace 来决定。我们看一下：

```
mysql> SHOW VARIABLES LIKE 'optimizer_trace';
+-----------------+--------------------------+
| Variable_name   | Value                    |
+-----------------+--------------------------+
| optimizer_trace | enabled=off,one_line=off |
+-----------------+--------------------------+
1 row in set (0.02 sec)
```

可以看到 enabled 的值为 off，表明这个功能默认是关闭的。

小贴士

one_line 的值用来控制输出格式。如果它的值为 on，那么所有输出都将在一行中展示。这不适合我们人类阅读，所以就保持其默认值为 off 吧。

如果想打开 optimizer trace 功能，必须首先把 enabled 的值改为 on，就像下面这样：

```
mysql> SET optimizer_trace="enabled=on";
Query OK, 0 rows affected (0.00 sec)
```

然后我们就可以输入想要查看优化过程的查询语句。当该查询语句执行完成后，就可以到 information_schema 数据库下的 OPTIMIZER_TRACE 表中查看完整的执行计划生成过程（也可以不真正执行查询语句，仅使用 EXPLAIN 来查看该语句的执行计划）。OPTIMIZER_TRACE 表有 4 列，分别如下。

- QUERY：表示我们输入的查询语句。
- TRACE：表示优化过程的 JSON 格式的文本。
- MISSING_BYTES_BEYOND_MAX_MEM_SIZE：在执行计划的生成过程中可能会输出很多内容，如果超过某个限制，多余的文本将不会显示。这个字段则展示了被忽略

的文本字节数。

- INSUFFICIENT_PRIVILEGES：表示是否有权限查看执行计划的生成过程，默认值是 0，表示有权限查看执行计划的生成过程；只有某些特殊情况下，它的值才是 1。我们暂时不关心这个字段的值。

使用 optimizer trace 功能的完整步骤如下所示。

步骤 1. 打开 optimizer trace 功能（默认情况下是关闭的）。

```
SET optimizer_trace="enabled=on";
```

步骤 2. 输入自己的查询语句。

```
SELECT ...;
```

步骤 3. 从 OPTIMIZER_TRACE 表中查看上一个查询的优化过程。

```
SELECT * FROM information_schema.OPTIMIZER_TRACE;
```

步骤 4. 可能还要观察其他语句执行的优化过程；重复步骤 2 和步骤 3。

步骤 5. 当停止查看语句的优化过程时，把 optimizer trace 功能关闭。

```
SET optimizer_trace="enabled=off";
```

现在我们有一个搜索条件比较多的查询语句，它的执行计划如下：

```
mysql> EXPLAIN SELECT * FROM s1 WHERE
    ->     key1 > 'z' AND
    ->     key2 < 1000000 AND
    ->     key3 IN ('a', 'b', 'c') AND
    ->     common_field = 'abc';
+----+-------------+-------+------------+-------+-----------------------+---------+---------+------+----------+-----------------------------+
| id | select_type | table | partitions | type  | possible_keys         | key     | key_len | ref  | rows     | filtered | Extra                       |
+----+-------------+-------+------------+-------+-----------------------+---------+---------+------+----------+-----------------------------+
| 1  | SIMPLE      | s1    | NULL       | range | uk_key2,idx_key1,idx_key3 | uk_key2 | 5       | NULL |   12     |    0.42  | Using index condition; Using where |
+----+-------------+-------+------------+-------+-----------------------+---------+---------+------+----------+-----------------------------+
1 row in set, 1 warning (0.00 sec)
```

可以看到，该查询可能使用到的索引有 3 个。为什么查询优化器最终选择了 uk_key2 而不选择其他的索引或者直接全表扫描呢？这时可以通过 otpimzer trace 功能来查看查询优化器的具体工作过程。

```
SET optimizer_trace="enabled=on";

SELECT * FROM s1 WHERE
    key1 > 'z' AND
    key2 < 1000000 AND
    key3 IN ('a', 'b', 'c') AND
    common_field = 'abc';

SELECT * FROM information_schema.OPTIMIZER_TRACE\G
```

16.2 通过 optimizer trace 分析查询优化器的具体工作过程

我们直接看一下通过查询 OPTIMIZER_TRACE 表得到的输出（这里在输出中使用了 # 后跟随注释的形式解释了优化过程中一些比较重要的点，请大家重点关注）。

```
*************************** 1. row ***************************
# 分析的查询语句是什么
QUERY: SELECT * FROM s1 WHERE
    key1 > 'z' AND
    key2 < 1000000 AND
    key3 IN ('a', 'b', 'c') AND
    common_field = 'abc'

# 优化的具体过程
TRACE: {
  "steps": [
    {
      "join_preparation": {     # prepare阶段
        "select#": 1,
        "steps": [
          {
            "IN_uses_bisection": true
          },
          {
            "expanded_query": "/* select#1 */ select 's1'.'id' AS 'id','s1'.'key1' AS 'key1',
's1'.'key2' AS 'key2','s1'.'key3' AS 'key3','s1'.'key_part1' AS 'key_part1','s1'.'key_part2'
AS 'key_part2','s1'.'key_part3' AS 'key_part3','s1'.'common_field' AS 'common_field' from 's1'
where (('s1'.'key1' > 'z') and ('s1'.'key2' < 1000000) and ('s1'.'key3' in ('a','b','c')) and
('s1'.'common_field' = 'abc'))"
          }
        ] /* steps */
      } /* join_preparation */
    },
    {
      "join_optimization": {    # optimize阶段
        "select#": 1,
        "steps": [
          {
            "condition_processing": {   # 处理搜索条件
              "condition": "WHERE",
              # 原始搜索条件
              "original_condition": "(('s1'.'key1' > 'z') and ('s1'.'key2' < 1000000) and
('s1'.'key3' in ('a','b','c')) and ('s1'.'common_field' = 'abc'))",
              "steps": [
                {
                  # 等值传递转换
                  "transformation": "equality_propagation",
                  "resulting_condition": "(('s1'.'key1' > 'z') and ('s1'.'key2' < 1000000)
and ('s1'.'key3' in ('a','b','c')) and ('s1'.'common_field' = 'abc'))"
                },
                {
                  # 常量传递转换
                  "transformation": "constant_propagation",
                  "resulting_condition": "(('s1'.'key1' > 'z') and ('s1'.'key2' < 1000000) and
('s1'.'key3' in ('a','b','c')) and ('s1'.'common_field' = 'abc'))"
                },
                {
                  # 去除没用的条件
```

```
                "transformation": "trivial_condition_removal",
                "resulting_condition": "(('s1'.'key1' > 'z') and ('s1'.'key2' < 1000000) and
('s1'.'key3' in ('a','b','c')) and ('s1'.'common_field' = 'abc'))"
              }
            ] /* steps */
          } /* condition_processing */
        },
        {
          # 替换虚拟生成列
          "substitute_generated_columns": {
          } /* substitute_generated_columns */
        },
        {
          # 表的依赖信息
          "table_dependencies": [
            {
              "table": "'s1'",
              "row_may_be_null": false,
              "map_bit": 0,
              "depends_on_map_bits": [
              ] /* depends_on_map_bits */
            }
          ] /* table_dependencies */
        },
        {
          "ref_optimizer_key_uses": [
          ] /* ref_optimizer_key_uses */
        },
        {

          # 预估不同单表访问方法的访问成本
          "rows_estimation": [
            {
              "table": "'s1'",
              "range_analysis": {
                "table_scan": {   # 全表扫描的行数以及成本
                  "rows": 9688,
                  "cost": 2036.7
                } /* table_scan */,

                # 分析可能使用的索引
                "potential_range_indexes": [
                  {
                    "index": "PRIMARY",   # 主键不可用
                    "usable": false,
                    "cause": "not_applicable"
                  },
                  {
                    "index": "uk_key2",  # uk_key2可能被使用
                    "usable": true,
                    "key_parts": [
                      "key2"
                    ] /* key_parts */
                  },
```

```
    {
      "index": "idx_key1",  # idx_key1可能被使用
      "usable": true,
      "key_parts": [
        "key1",
        "id"
      ] /* key_parts */
    },
    {
      "index": "idx_key3",  # idx_key3可能被使用
      "usable": true,
      "key_parts": [
        "key3",
        "id"
      ] /* key_parts */
    },
    {
      "index": "idx_key_part",  # idx_keypart不可用
      "usable": false,
      "cause": "not_applicable"
    }
  ] /* potential_range_indexes */,
  "setup_range_conditions": [
  ] /* setup_range_conditions */,
  "group_index_range": {
    "chosen": false,
    "cause": "not_group_by_or_distinct"
  } /* group_index_range */,

  # 分析各种可能使用的索引的成本
  "analyzing_range_alternatives": {
    "range_scan_alternatives": [
      {
        # 使用uk_key2的成本分析
        "index": "uk_key2",
        # 使用uk_key2的扫描区间
        "ranges": [
          "NULL < key2 < 1000000"
        ] /* ranges */,
        "index_dives_for_eq_ranges": true,    # 是否使用index dive
        "rowid_ordered": false,      # 使用该索引获取的记录是否按照主键排序
        "using_mrr": false,      # 是否使用mrr
        "index_only": false,     # 是否是覆盖索引
        "rows": 12,      # 使用该索引获取的记录条数
        "cost": 15.41,   # 使用该索引的成本
        "chosen": true   # 是否选择该索引
      },
      {
        # 使用idx_key1的成本分析
        "index": "idx_key1",
        # 使用idx_key1的扫描区间
        "ranges": [
          "z < key1"
        ] /* ranges */,
```

```
            "index_dives_for_eq_ranges": true,   # 同上
            "rowid_ordered": false,   # 同上
            "using_mrr": false,   # 同上
            "index_only": false,   # 同上
            "rows": 266,   # 同上
            "cost": 320.21,   # 同上
            "chosen": false,   # 同上
            "cause": "cost"   # 因成本太大而不选择该索引
          },
          {
            # 使用idx_key3的成本分析
            "index": "idx_key3",
            # 使用idx_key3的扫描区间
            "ranges": [
              "a <= key3 <= a",
              "b <= key3 <= b",
              "c <= key3 <= c"
            ] /* ranges */,
            "index_dives_for_eq_ranges": true,   # 同上
            "rowid_ordered": false,   # 同上
            "using_mrr": false,   # 同上
            "index_only": false,   # 同上
            "rows": 21,   # 同上
            "cost": 28.21,   # 同上
            "chosen": false,   # 同上
            "cause": "cost"   # 同上
          }
        ] /* range_scan_alternatives */,

        # 分析使用索引合并的成本
        "analyzing_roworder_intersect": {
          "usable": false,
          "cause": "too_few_roworder_scans"
        } /* analyzing_roworder_intersect */
      } /* analyzing_range_alternatives */,

      # 对于上述单表查询s1最优的访问方法
      "chosen_range_access_summary": {
        "range_access_plan": {
          "type": "range_scan",
          "index": "uk_key2",
          "rows": 12,
          "ranges": [
            "NULL < key2 < 1000000"
          ] /* ranges */
        } /* range_access_plan */,
        "rows_for_plan": 12,
        "cost_for_plan": 15.41,
        "chosen": true
      } /* chosen_range_access_summary */
    } /* range_analysis */
  }
] /* rows_estimation */
},
```

```
{
  # 分析各种可能的执行计划
  #（对于多表查询，可能有很多种不同的方案：单表查询的方案已经在前面分析过了，直接选取uk_key2就好）
  "considered_execution_plans": [
    {
      "plan_prefix": [
      ] /* plan_prefix */,
      "table": "'s1'",
      "best_access_path": {
        "considered_access_paths": [
          {
            "rows_to_scan": 12,
            "access_type": "range",
            "range_details": {
              "used_index": "uk_key2"
            } /* range_details */,
            "resulting_rows": 12,
            "cost": 17.81,
            "chosen": true
          }
        ] /* considered_access_paths */
      } /* best_access_path */,
      "condition_filtering_pct": 100,
      "rows_for_plan": 12,
      "cost_for_plan": 17.81,
      "chosen": true
    }
  ] /* considered_execution_plans */
},
{
  # 尝试给查询添加一些其他的查询条件
  "attaching_conditions_to_tables": {
    "original_condition": "(('s1'.'key1' > 'z') and ('s1'.'key2' < 1000000) and
('s1'.'key3' in ('a','b','c')) and ('s1'.'common_field' = 'abc'))",
    "attached_conditions_computation": [
    ] /* attached_conditions_computation */,
    "attached_conditions_summary": [
      {
        "table": "'s1'",
        "attached": "(('s1'.'key1' > 'z') and ('s1'.'key2' < 1000000) and
('s1'.'key3' in ('a','b','c')) and ('s1'.'common_field' = 'abc'))"
      }
    ] /* attached_conditions_summary */
  } /* attaching_conditions_to_tables */
},
{
  # 再稍稍改进一下执行计划
  "refine_plan": [
    {
      "table": "'s1'",
      "pushed_index_condition": "('s1'.'key2' < 1000000)",
      "table_condition_attached": "(('s1'.'key1' > 'z') and ('s1'.'key3' in
('a','b','c')) and ('s1'.'common_field' = 'abc'))"
```

```
            }
          ] /* refine_plan */
        }
      ] /* steps */
    } /* join_optimization */
  },
  {
    "join_execution": {     # execute阶段
      "select#": 1,
      "steps": [
      ] /* steps */
    } /* join_execution */
  }
] /* steps */
}

# 因优化过程文本太多而丢弃的文本字节大小，值为0时表示并没有丢弃
MISSING_BYTES_BEYOND_MAX_MEM_SIZE: 0

# 权限字段
INSUFFICIENT_PRIVILEGES: 0

1 row in set (0.00 sec)
```

大家看到这个输出的第一感觉应该就是“这内容也太多了吧”。其实这只是查询优化器执行过程中的一小部分，设计 MySQL 的大叔可能会在之后的版本中添加更多的优化过程信息。这些信息看似杂乱，其实还是很有规律的。优化过程大致分为了 3 个阶段：

- prepare 阶段；
- optimize 阶段；
- execute 阶段。

我们所说的基于成本的优化主要集中在 optimize 阶段。对于单表查询来说，主要关注的是 optimize 阶段的 rows_estimation 过程。这个过程深入分析了针对单表查询的各种执行方案的成本；对于多表连接查询来说，我们更多关注的是 considered_execution_plans 过程。这个过程中会写明各种不同的表连接顺序所对应的成本。反正查询优化器最终会选择成本最低的方案来作为最终的执行计划，即我们使用 EXPLAIN 语句所展现出的那种方案。

如果有同学对使用 EXPLAIN 语句展示出的针对某个查询的执行计划很不理解，可以尝试使用 optimizer trace 功能详细了解每一种执行方案对应的成本。相信这个功能能让大家更深入地了解 MySQL 查询优化器。

小贴士

上文介绍的 rows_estimation 过程分析的其实都是 range 访问方法对应的成本，并没有涉及 ref 访问方法。对于 ref 访问方法来说，在计算回表操作的 I/O 成本时存在天花板（第 12 章中有提及）。ref 访问方法所对应的成本是被单独计算的，计算过程体现在 considered_execution_plans->best_access_path->considered_access_paths 中，本例中没有使用 ref 访问方法执行查询的场景，在 optimizer trace 的输出中并未体现。